

```

# --- Setup ---
!pip install qiskit qiskit-aer matplotlib --quiet

# --- Imports ---
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, pauli_error

# --- QCC Echo Kernel ---
def build_kernel():
    qc = QuantumCircuit(3, 3)
    qc.h(0)
    qc.cx(0, 1)
    qc.cx(1, 2)
    qc.cx(1, 2)
    qc.cx(0, 1)
    qc.h(0)
    qc.barrier()
    qc.delay(200, 0, 'ns')
    qc.delay(200, 1, 'ns')
    qc.delay(200, 2, 'ns')
    qc.measure([0, 1, 2], [0, 1, 2])
    return qc

def compute_entropy_and_fidelity(counts):
    probs = np.array(list(counts.values())) / sum(counts.values())
    entropy = -np.sum([p * np.log2(p) for p in probs if p > 0])
    fidelity = max(probs) * 100
    return entropy, fidelity

# --- Sweep Values ---
x_vals = np.linspace(0.0, 0.4, 9)
y_vals = np.linspace(0.0, 0.4, 9)
i_vals = np.linspace(0.0, 0.4, 9)

# --- Allocate Result Storage ---
entropy_map = np.zeros((len(x_vals), len(y_vals), len(i_vals)))
fidelity_map = np.zeros((len(x_vals), len(y_vals), len(i_vals)))

# --- Full 3D Sweep Over X, Y, I ---
for i, x in enumerate(x_vals):
    for j, y in enumerate(y_vals):
        for k, i_prob in enumerate(i_vals):
            total = x + y + i_prob
            if total > 1.0:
                # Normalize to sum to 1.0 if overflow
                x_n = x / total
                y_n = y / total
                i_n = i_prob / total
            else:
                x_n = x
                y_n = y
                i_n = 1.0 - (x + y) # Automatically adjust I if room

            # Build Noise Model
            error_1q = pauli_error([('X', x_n), ('Y', y_n), ('I', i_n)])

```

```

error_2q = error_1q.tensor(error_1q)
noise_model = NoiseModel()
noise_model.add_all_qubit_quantum_error(error_2q, ['cx'])

# Simulate
qc = build_kernel()
sim = AerSimulator(noise_model=noise_model)
result = sim.run(transpile(qc, sim), shots=8192).result()
counts = result.get_counts()
entropy, fidelity = compute_entropy_and_fidelity(counts)

entropy_map[i, j, k] = entropy
fidelity_map[i, j, k] = fidelity

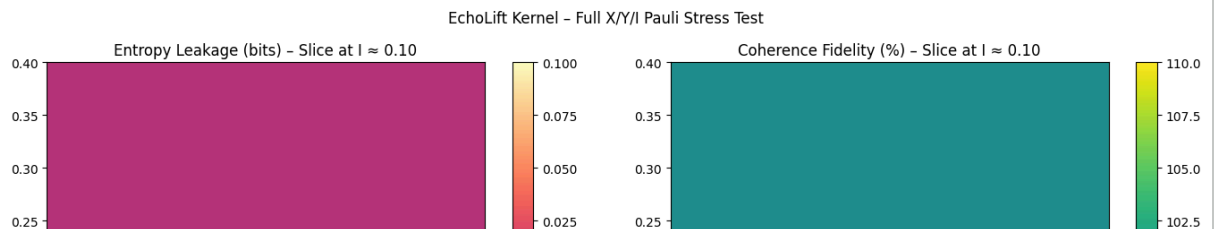
# --- Plot: Choose a slice at a fixed I level ---
i_index = 2 # Choose I ≈ 0.1 (adjustable)
fig, axs = plt.subplots(1, 2, figsize=(14, 6))

im1 = axs[0].imshow(entropy_map[:, :, i_index], origin='lower',
                    extent=[0, 0.4, 0, 0.4], aspect='auto', cmap='magma')
axs[0].set_title("Entropy Leakage (bits) - Slice at I ≈ {:.2f}".format(i_vals[i_index]))
axs[0].set_xlabel("Y Probability")
axs[0].set_ylabel("X Probability")
fig.colorbar(im1, ax=axs[0])

im2 = axs[1].imshow(fidelity_map[:, :, i_index], origin='lower',
                    extent=[0, 0.4, 0, 0.4], aspect='auto', cmap='viridis')
axs[1].set_title("Coherence Fidelity (%) - Slice at I ≈ {:.2f}".format(i_vals[i_index]))
axs[1].set_xlabel("Y Probability")
axs[1].set_ylabel("X Probability")
fig.colorbar(im2, ax=axs[1])

plt.suptitle("EchoLift Kernel - Full X/Y/I Pauli Stress Test")
plt.tight_layout()
plt.show()

```



```

# --- Setup ---
!pip install qiskit qiskit-aer matplotlib --quiet

# --- Imports ---
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, pauli_error

# --- QCC Echo Kernel ---
def build_kernel():
    qc = QuantumCircuit(3, 3)
    qc.h(0)

```

```

qc.cx(0, 1)
qc.cx(1, 2)
qc.cx(1, 2)
qc.cx(0, 1)
qc.h(0)
qc.barrier()
qc.delay(200, 0, 'ns')
qc.delay(200, 1, 'ns')
qc.delay(200, 2, 'ns')
qc.measure([0, 1, 2], [0, 1, 2])
return qc

def compute_entropy_and_fidelity(counts):
    probs = np.array(list(counts.values())) / sum(counts.values())
    entropy = -np.sum([p * np.log2(p) for p in probs if p > 0])
    fidelity = max(probs) * 100
    return entropy, fidelity

# --- Sweep Values ---
x_vals = np.linspace(0.3, 0.9, 7)
y_vals = np.linspace(0.3, 0.9, 7)
i_vals = np.linspace(0.0, 0.4, 5)

# --- Allocate Result Storage ---
entropy_map = np.zeros((len(x_vals), len(y_vals), len(i_vals)))
fidelity_map = np.zeros((len(x_vals), len(y_vals), len(i_vals)))

# --- Full 3D Sweep Over X, Y, I ---
for i, x in enumerate(x_vals):
    for j, y in enumerate(y_vals):
        for k, i_prob in enumerate(i_vals):
            total = x + y + i_prob
            if total > 1.0:
                # Normalize to sum to 1.0 if overflow
                x_n = x / total
                y_n = y / total
                i_n = i_prob / total
            else:
                x_n = x
                y_n = y
                i_n = 1.0 - (x + y) # Automatically adjust I if room

            # Build Noise Model
            error_1q = pauli_error([('X', x_n), ('Y', y_n), ('I', i_n)])
            error_2q = error_1q.tensor(error_1q)
            noise_model = NoiseModel()
            noise_model.add_all_qubit_quantum_error(error_2q, ['cx'])

            # Simulate
            qc = build_kernel()
            sim = AerSimulator(noise_model=noise_model)
            result = sim.run(transpile(qc, sim), shots=8192).result()
            counts = result.get_counts()
            entropy, fidelity = compute_entropy_and_fidelity(counts)

            entropy_map[i, j, k] = entropy
            fidelity_map[i, j, k] = fidelity

# --- Plot: Choose a slice at a fixed I level ---
i_index = 2 # Choose I ≈ 0.1 (adjustable)

```

```

fig, axs = plt.subplots(1, 2, figsize=(14, 6))

im1 = axs[0].imshow(entropy_map[:, :, i_index], origin='lower',
                    extent=[0, 0.4, 0, 0.4], aspect='auto', cmap='magma')
axs[0].set_title("Entropy Leakage (bits) - Slice at I ≈ {:.2f}".format(i_vals[i_index]))
axs[0].set_xlabel("Y Probability")
axs[0].set_ylabel("X Probability")
fig.colorbar(im1, ax=axs[0])

im2 = axs[1].imshow(fidelity_map[:, :, i_index], origin='lower',
                    extent=[0, 0.4, 0, 0.4], aspect='auto', cmap='viridis')
axs[1].set_title("Coherence Fidelity (%) - Slice at I ≈ {:.2f}".format(i_vals[i_index]))
axs[1].set_xlabel("Y Probability")
axs[1].set_ylabel("X Probability")
fig.colorbar(im2, ax=axs[1])

plt.suptitle("EchoLift Kernel - Full X/Y/I Pauli Stress Test")
plt.tight_layout()
plt.show()

```

```
!pip install ipynbname
```

```

Requirement already satisfied: ipynbname in /usr/local/lib/python3.12/dist
Requirement already satisfied: ipykernel in /usr/local/lib/python3.12/dist
Requirement already satisfied: debugpy>=1.0 in /usr/local/lib/python3.12/
Requirement already satisfied: ipython>=7.23.1 in /usr/local/lib/python3.
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/p
Requirement already satisfied: matplotlib-inline>=0.1 in /usr/local/lib/p
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.12/
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dis
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pyzmq>=17 in /usr/local/lib/python3.12/dis
Requirement already satisfied: tornado>=6.1 in /usr/local/lib/python3.12/
Requirement already satisfied: traitlets>=5.1.0 in /usr/local/lib/python3
Requirement already satisfied: setuptools>=18.5 in /usr/local/lib/python3
Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.12/di
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dis
Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/d
Requirement already satisfied: prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0
Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist
Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/d
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/d
Requirement already satisfied: jupyter-core>=4.9.2 in /usr/local/lib/pyth
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/p
Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/pyth
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python
Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist

```

```

"""
-----BEGIN QTET SIGNED HASH-----
Validation SHA-256:
Notebook: Untitled13.ipynb

```

SHA-256: a85e1647fe9227ecbf927c74e8ed206767051236e4d95b56bef61428e128c9e7

Notebook: Pauli.ipynb

SHA-256: 19f7000c0241ba660dadab63ca36f97a0f00e7a2453f43c6e13d6f6ca6d3a385

UTC Timestamp: 2025-09-05T18:55:52.018160Z

Signed with intent by:

Destiny Machwaya

echoarc.research@proton.me

This signature affirms authorship and cryptographic traceability,
and will be upgraded to GPG-formal signature once secure key generation is available.

-----END QTET SIGNED HASH-----

""

'\n-----BEGIN QTET SIGNED HASH-----\nValidation SHA-256:\nNotebook: Untitled13.ipynb\nSHA-256: a85e1647fe9227ecbf927c74e8ed206767051236e4d95b56bef61428e128c9e7\n\nNotebook: Pauli.ipynb\nSHA-256: 19f7000c0241ba660dadab63ca36f97a0f00e7a2453f43c6e13d6f6ca6d3a385\n\nUTC Timestamp: 2025-09-05T18:55:52.018160Z\n\nSigned with intent by:\nDestiny Machwaya\nechoarc.research@proton.me\n\nThis signature affirms authorship and cryptographic traceability,\nand will be upgraded to GPG-formal signature once secure key generation is available.\n\n-----END QTET SIGNED HASH-----\n'

